

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE MATEMÁTICA E ESTATÍSTICA
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

**Modelo de Atores fora de sistemas
distribuídos**
*aplicando o padrão arquitetural em
aplicações centralizadas*

Jorge Harrisonn Mantovanelli Thomes Vieira

MONOGRAFIA FINAL
MAC 499 — TRABALHO DE
FORMATURA SUPERVISIONADO

Supervisor: Paulo Meirelles
Cossupervisor: David Tadokoro

São Paulo
2025

*O conteúdo deste trabalho é publicado sob a licença CC BY 4.0
(Creative Commons Attribution 4.0 International License)*

*Dedicado a meus avós Elizete e
Jorge e à minha finada mãe Sarah.*

[illegible]

Resumo

Jorge Harrisonn Mantovanelli Thomes Vieira. **Modelo de Atores fora de sistemas distribuídos: aplicando o padrão arquitetural em aplicações centralizadas**. Monografia (Bacharelado). Instituto de Matemática e Estatística, Universidade de São Paulo, São Paulo, 2025.

O

Palavras-chave: .

Modelo de Atores é uma teoria matemática largamente usada na arquitetura de sistemas distribuídos. Este trabalho visa explorar o Modelo de Atores aplicado no desenvolvimento de uma aplicação centralizada (isto é, não distribuída). Os objetivos são: propor uma arquitetura baseada no Modelo de Atores para aplicações centralizadas; implementar uma aplicação usando a arquitetura proposta; fazer uma análise qualitativa da aplicação produzida. A aplicação a ser desenvolvida será avaliada do ponto de vista de: testabilidade, performance, manutenibilidade e extensibilidade.

Palavras-chave: Modelo de Atores, Engenharia de Software, Arquitetura de Software, Rust, Aplicações Centralizadas

Abstract

Jorge Harrisonn Mantovanelli Thomes Vieira. **Actor Model outside distributed systems: *applying the architectural pattern to centralized applications***. Capstone Project Report (Bachelor). Institute of Mathematics and Statistics, University of São Paulo, São Paulo, 2025.

T

Keywords: .

The Actor Model is a mathematical theory widely used in distributed systems architecture. This work aims to explore the Actor Model applied to the development of a centralized application (i.e., not distributed). The objectives are: to propose an Actor Model-based architecture for centralized applications; to implement an application using the proposed architecture; to perform a qualitative analysis of the produced application. The application to be developed will be evaluated from the perspective of: testability, performance, maintainability and extensibility.

Keywords: Actor Model, Software Engineering, Software Architecture, Rust, Centralized Applications

Lista de abreviaturas

CFT	Transformada contínua de Fourier (<i>Continuous Fourier Transform</i>)
DFT	Transformada discreta de Fourier (<i>Discrete Fourier Transform</i>)
EIIP	Potencial de interação elétron-íon (<i>Electron-Ion Interaction Potentials</i>)
STFT	Transformada de Fourier de tempo reduzido (<i>Short-Time Fourier Transform</i>)
ABNT	Associação Brasileira de Normas Técnicas
URL	Localizador Uniforme de Recursos (<i>Uniform Resource Locator</i>)
IME	Instituto de Matemática e Estatística
USP	Universidade de São Paulo

Lista de símbolos

ω	Frequência angular
ψ	Função de análise <i>wavelet</i>
Ψ	Transformada de Fourier de ψ

Lista de figuras

Lista de tabelas

Lista de programas

B.1	Máximo divisor comum (arquivo importado).	53
B.2	Máximo divisor comum (em português).	53

Sumário

1	Introdução	1
2	Arquitetura de Software	3
2.1	Padrões Arquiteturais	3
2.1.1	Princípios SOLID	4
2.1.2	Padrão MVC	4
2.2	Avaliação de Adequação de Padrões	5
2.3	Justificativa para o Modelo de Atores	5
3	Modelo de Atores	7
3.1	O que é um Ator?	7
3.2	Indeterminação	8
3.3	Implementações Conhecidas	8
3.3.1	A Linguagem de Programação Elixir	9
3.3.2	Exemplo em Elixir	11
4	Proposta de Modelos de Atores em Rust	15
4.1	Rust	15
4.1.1	Segurança de Memória	16
4.1.2	Sistema de Tipos	17
4.2	Concorrência e Paralelismo	19
4.3	Compartilhamento de Memória	20
4.4	Passagem de Mensagens e Endereços	21
4.4.1	<code>mpsc</code> - <i>Multiple producer, single consumer</i>	21
4.4.2	<code>oneshot</code>	22
4.5	Representação do Ator	22
4.5.1	Mensagens	23
4.5.2	Lógica	23
4.5.3	Endereço	25

4.6	Tratamento de Erros	26
4.6.1	Hierarquia de Erros	27
4.6.2	Erros Fatais	27
4.6.3	Erros Específicos de Ator	28
4.6.4	Propagação de Erros através de Canais	28
4.6.5	Tratamento de Erros na Interface Pública	29
4.6.6	Erros Recuperáveis e Não-Recuperáveis	29
4.6.7	Funções Auxiliares para Criação de Erros	30
4.7	Testabilidade e Mocks	30
4.7.1	Mockall	30
4.7.2	Integração de Mocks	31
4.7.3	Testes Unitários	31
5	Estudo de Caso: Patch-Hub	33
5.1	Contexto	33
5.1.1	O Processo de Contribuição para o Kernel Linux	33
5.1.2	O Projeto lore.kernel.org	33
5.1.3	O Projeto KWorkflow	34
5.1.4	O Patch-Hub	34
5.2	Arquitetura Original	34
5.3	Funcionalidades Principais	35
5.4	Limitações da Arquitetura Original	35
5.4.1	Gargalo de Mutação Centralizada	35
5.4.2	Impossibilidade de Composição Modular	36
5.4.3	Atualizações de Estado Intercomponentes Problemáticas	36
5.4.4	Concorrência e Responsividade Limitadas	36
5.4.5	Dificuldades de Testabilidade	37
5.5	Objetivos	37
5.6	Modelagem com Atores	38
5.6.1	Atores de Integração	38
5.6.2	Atores de Lógica	40
6	Conclusão	47
A	Perguntas frequentes sobre o modelo	49
A	As packages imegoodies e imelooks	51
B	Código-fonte e pseudocódigo	53

Referências	55
Índice remissivo	57

Capítulo 1

Introdução

Capítulo 2

Arquitetura de Software

Todo sistema é desenvolvido com um propósito. A forma de saber se o sistema está atendendo ao seu propósito é estabelecendo requisitos que impõem restrições e direcionam o processo de desenvolvimento para a direção correta. Os requisitos em si são uma grande área de pesquisa, mas podem ser principalmente divididos em dois grupos: funcionais e não funcionais.

Requisitos funcionais são restrições que especificam o que o sistema deve ser capaz de fazer. Um requisito funcional para uma rede social pode ser a capacidade de postar fotos. Por outro lado, **requisitos não funcionais**, ou requisitos de qualidade, são restrições que definem como algo deve ser feito. Um requisito não funcional para a rede social mencionada pode ser o armazenamento seguro das fotos dos usuários, impedindo acesso não autorizado a elas. Em resumo, requisitos funcionais definem funcionalidades, enquanto os não funcionais estabelecem expectativas de qualidade.

2.1 Padrões Arquiteturais

A **arquitetura de software** é definida como um campo de pesquisa que busca entender como decisões de design afetam a estrutura, comportamento e qualidade geral de um sistema de software, servindo como base para decisões subsequentes. É o campo que lança luz sobre a definição de requisitos de qualidade e como atender a eles.

Muitas decisões de design estudadas pela arquitetura de software podem ser agrupadas de acordo com a categoria de problema que resolvem. Desse modo, se cria um arcabouço de conceitos bem estabelecidos reutilizáveis chamados de **padrões arquiteturais** (daqui em diante simplesmente referidos como “padrões”).

Padrões tem por objetivo garantir a qualidade do software oferecendo soluções para atender aos requisitos não funcionais de sistemas. Isso significa que podem ser aplicados a qualquer sistema, independentemente de suas funcionalidades, desde que tenham requisitos de qualidade similares.

Todavia, como explorado pelo artigo *Software Patterns in Practice: an Empirical Study*, em geral, na indústria, na concepção dos projetos, as funcionalidades são mais levadas em

conta do que os requisitos de qualidade. O mesmo artigo ainda afirma que se funcionalidades fossem a única parte importante, não haveria necessidade de ter arquitetura de software.

Por isso, é necessário estudar e entender os padrões existentes e como isso traz consequências de longo prazo para confiabilidade, manutenibilidade, testabilidade, entre outras complicações para um sistema.

A seguir, apresentam-se alguns padrões arquiteturais fundamentais que servirão como base conceitual para as discussões subsequentes neste trabalho.

2.1.1 Princípios SOLID

Os princípios SOLID são um conjunto de diretrizes para o design de software introduzidos por Robert Martin no ano 2000. Originalmente, os princípios tinham como alvo principal sistemas orientados a objetos, todavia suas lições podem se estender muito além e ser aplicados a diversos domínios. Seus principais objetivos são criar sistemas mais flexíveis, manuteníveis e extensíveis. Seu nome, é uma sigla com as iniciais de cada um dos 5 princípios, que são:

- **Single Responsibility:** Uma classe deve ter apenas uma razão para mudar, ou seja, deve ter apenas uma responsabilidade.
- **Open/Closed:** Entidades de software devem estar abertas para extensão, mas fechadas para modificação.
- **Liskov Substitution:** Objetos de uma superclasse devem ser substituíveis por objetos de suas subclasses sem alterar a funcionalidade do programa.
- **Interface Segregation:** Clientes não devem ser forçados a depender de interfaces que não utilizam.
- **Dependency Inversion:** Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações.

Estes princípios são especialmente relevantes para o Modelo de Atores, pois cada ator pode ser visto como uma unidade que segue o princípio de responsabilidade única, enquanto a comunicação baseada em mensagens facilita a inversão de dependências e a segregação de interfaces.

2.1.2 Padrão MVC

O padrão Model-View-Controller (MVC) é um padrão arquitetural que divide uma aplicação em três componentes interconectados, cada um com responsabilidades bem definidas. Esse padrão surgiu para resolver problemas de acoplamento entre a lógica de negócio e a apresentação, permitindo que essas camadas evoluam de forma independente.

Os três componentes do padrão MVC são:

- **Model:** Representa os dados e a lógica de negócio da aplicação. É responsável por manter o estado do sistema e realizar operações sobre os dados. O Model não possui conhecimento sobre como os dados são exibidos.

- **View:** É responsável pela apresentação dos dados ao usuário. A View consome dados do Model e os exibe de forma adequada à interface (web, desktop, terminal, etc.). A View não deve conter lógica de negócio complexa.
- **Controller:** Funciona como intermediário entre a View e o Model. Recebe entradas do usuário através da View, processa essas entradas (frequentemente chamando operações no Model) e atualiza o estado da aplicação. O Controller orquestra a comunicação entre os outros dois componentes.

O MVC oferece benefícios significativos para a manutenibilidade e testabilidade de sistemas. A separação de responsabilidades permite que a lógica de negócio seja testada independentemente da interface, facilitando testes unitários. Além disso, múltiplas Views podem ser criadas para consumir o mesmo Model, permitindo reutilização de código e adaptação a diferentes contextos.

No entanto, o padrão MVC tradicional apresenta algumas limitações em contextos específicos. Em aplicações com requisitos complexos de concorrência ou que precisam lidar com múltiplas operações assíncronas, o Controller pode se tornar um ponto de gargalo. Além disso, em aplicações distribuídas, a comunicação síncrona típica do MVC pode não ser suficiente para atender aos requisitos de resiliência e escalabilidade.

2.2 Avaliação de Adequação de Padrões

O primeiro e mais importante fator a ser analisado para se identificar a adequação de um padrão para um projeto é, evidentemente, o casamento entre os requisitos de qualidade que o padrão aborda e o requisitos de qualidade do sistema.

Uma outra característica a ser considerada é o contexto da aplicação. Existem padrões que abordam desafios comuns para aplicações de nuvem, outros para aplicações de interface gráfica. Escolher um padrão de um domínio distinto irá fornecer poucas ferramentas úteis para a resolução do problema.

Por fim, é necessário entender os custos que aquele padrão trás. Qual impacto esperado na performance do sistema? O quanto ele enrijece a estrutura pro projeto? O quão mais complicado fica a organização do código? Todas essas perguntas precisam ser feitas.

2.3 Justificativa para o Modelo de Atores

O propósito deste trabalho é experimentar com um padrão arquitetural específico chamado Modelo de Atores. A ideia é usar este padrão em um projeto que está bem distante dos requisitos de qualidade que ele tradicionalmente atende e entender quais são as consequências.

Esta abordagem experimental permitirá:

- Explorar os limites do Modelo de Atores em contextos não distribuídos
- Avaliar se os benefícios do padrão se mantêm em aplicações centralizadas

- Identificar possíveis adaptações necessárias para diferentes domínios
- Contribuir para o conhecimento sobre a aplicabilidade de padrões arquiteturais

Capítulo 3

Modelo de Atores

O Modelo de Atores de computação foi estabelecido por Carl Hewitt em 1973, baseado em trabalhos anteriores de Peter Bishop e Richard Steiger. Seus objetivos eram abordar os novos desafios impostos pelos sistemas massivamente concorrentes e paralelos que emergiram com a computação em nuvem e arquiteturas many-core.

Seus objetivos foram alcançados criando um modelo computacional que é inerentemente concorrente. Uma grande mudança dos modelos anteriores é que ele é baseado em leis físicas, ao invés de álgebra pura. A hipótese mais importante defendida pelo trabalho de Hewitt é:

Toda computação fisicamente possível pode ser diretamente implementada com Atores

O Modelo de Atores representa uma reformulação completa da computação concorrente, passando de máquinas de estado globais para sistemas distribuídos, assíncronos e baseados em passagem de mensagens que melhor refletem a realidade física dos sistemas de computação modernos.

3.1 O que é um Ator?

Hewitt propõe este primitivo chamado Ator que é identificado por um endereço. Atores são entidades que se comunicarão entre si através de mensagens. Uma vez que uma mensagem é recebida, um Ator pode:

- Enviar mensagens para Atores cujos endereços conhece
- Criar novos Atores
- Modificar seu próprio comportamento para lidar com mensagens futuras recebidas

Mensagens são a unidade de comunicação e são passadas de forma assíncrona entre atores. De acordo com o princípio de segurança e localidade, ao processar uma mensagem recebida, um Ator pode apenas enviar mensagens para Atores cujos endereços foram:

- Conhecidos anteriormente

- Parte da mensagem recebida
- Criados durante o processamento atual

3.2 Indeterminação

Aqui vem um dos princípios fundamentais do Modelo de Atores e o que o diferencia de outros padrões. A ordem de chegada das mensagens não é garantida e o Modelo de Atores não tentará resolver isso, mas sim tratar isso como uma propriedade natural do domínio. O Modelo de Atores é projetado para sistemas distribuídos, o que significa que dois atores podem estar se comunicando através de longas distâncias físicas.

Isso significa que as mensagens enviadas podem levar tempo ilimitado para chegar na máquina destino, e mesmo que cheguem exatamente ao mesmo tempo, os árbitros de hardware que as ordenarão não garantirão nenhum tipo de ordenação. Considerar os fatores físicos é o que diferencia o Modelo de Atores de padrões de sistemas concorrentes puramente algébricos.

A ordenação indeterminada de mensagens, somada ao fato de que as mensagens podem alterar o comportamento de um Ator, tem por consequência que as mesmas condições iniciais podem resultar em resultados diferentes. Além disso, o próprio sistema é dito não ter um estado global definido. Como a comunicação é assíncrona, a qualquer momento é possível ter mensagens em trânsito ou Atores em estado transitório não sendo possível garantir que em algum momento todos os atores estarão em estados bem definidos.

Isso é o que garante a justiça (fairness).

3.3 Implementações Conhecidas

O Modelo de Atores recebeu a devida atenção e saiu do estado de um modelo/teoria computacional para se transformar em um padrão arquitetural com foco em abordar sistemas distribuídos. Bibliotecas e arcabouços como Akka (Java) e Orleans (C#) fornecem suporte ao Modelo de Atores para linguagens de programação. No entanto, a implementação mais proeminente e bem-sucedida é a linguagem de programação Elixir.

Na década de 1980, a empresa de telecomunicações sueca Ericsson experimentou com linguagens de programação funcional para abordar desafios de telecomunicações, desenvolvendo eventualmente Erlang. Erlang focou em concorrência, distribuição e tolerância a falhas implementando o Modelo de Atores no nível da linguagem. BEAM, a máquina abstrata que serviu como runtime para programas Erlang, representou a maior conquista da Ericsson neste domínio. (Na época, o termo "máquina virtual" ainda não era aplicado a tais sistemas.) No entanto, a sintaxe do Erlang era considerada complexa, criando uma barreira para adoção mais ampla.

No início dos anos 2010, José Valim estava desenvolvendo soluções para sistemas distribuídos, os mesmos desafios que a Ericsson havia abordado três décadas antes. Ao descobrir Erlang e BEAM, ele reconheceu sua eficácia, mas os achou difíceis de usar. Isso

levou à criação de Elixir, que combinou o poder do BEAM e a flexibilidade do Modelo de Atores com uma sintaxe mais acessível inspirada em Ruby.

Ao contrário de implementações baseadas em bibliotecas que adicionam suporte ao Modelo de Atores a linguagens existentes, Elixir torna o Modelo de Atores o paradigma central da linguagem. Isso é alcançado através de suporte nativo para green threads, tolerância a falhas e comunicação inter-processo remota. Esses são diferenciadores que estabeleceram Elixir como a principal plataforma do Modelo de Atores.

Esta linguagem dinâmica continua ganhando reconhecimento, sendo eleita como a segunda linguagem de programação mais admirada por três anos consecutivos na Pesquisa de Desenvolvedores do Stack Overflow. Portanto, este trabalho usará Elixir como a implementação de referência e exemplo de estado da arte do Modelo de Atores.

3.3.1 A Linguagem de Programação Elixir

A filosofia de design do Elixir difere fundamentalmente das linguagens orientadas a objetos. Ao invés de classes instanciadas em objetos, Elixir usa módulos que geram atores por design. Esta abordagem torna o Modelo de Atores o paradigma central da linguagem.

Os atores no Elixir são representados por processos Erlang (daqui em diante referidos simplesmente como processos). Estes processos leves são criados usando a função `spawn`, que recebe três parâmetros: um módulo, um nome de função daquele módulo e uma lista de argumentos. Quando chamada, `spawn` executa a função especificada em um processo dedicado a ele e retorna um *Process Identifier* (PID) que serve como o endereço do ator.

A comunicação entre atores acontece através de passagem de mensagens usando duas funções: `send` para despachar mensagens para um processo usando seu PID como referência, e `receive` para lidar com mensagens recebidas dentro de um processo. Este mecanismo de passagem de mensagens assíncrono forma a base das capacidades concorrentes e distribuídas do Elixir.

Como Elixir é construída extensivamente sobre o ecossistema do Erlang e executa na máquina virtual BEAM, referências tanto ao Erlang quanto ao BEAM aparecerão frequentemente ao longo desta discussão.

O Modelo de Atores em Elixir

A implementação do Modelo de Atores em Elixir é simples, requerendo esforço mínimo para criar sistemas de atores concorrentes e distribuídos. O design da linguagem torna a programação baseada em atores natural ao invés de um padrão adicional a aprender.

Mensagens em Elixir são representadas por qualquer estrutura de dados válida: átomos, strings, números, tuplas, listas, ou mesmo estruturas aninhadas complexas. Esta flexibilidade permite que desenvolvedores projetem protocolos de mensagem que melhor se adequem às necessidades de sua aplicação sem serem restringidos por formatos de mensagem rígidos.

Endereços são *Process Identifiers* do Erlang (PIDs), que são identificadores únicos automaticamente gerados pela máquina virtual BEAM quando um processo é gerado. Estes

PIDs servem como o endereço do ator, permitindo roteamento de mensagens tanto em ambientes locais quanto distribuídos.

Manipulação de mensagens ocorre através da função `receive`, que permite que um processo aguarde o recebimento de uma mensagem, faça casamento de padrões. Este mecanismo permite que atores processem diferentes tipos de mensagens com lógica de manipulação apropriada, tornando o sistema tanto robusto quanto expressivo.

Noções Básicas do Elixir

Elixir é uma linguagem de programação dinamicamente tipada, o que significa que tipos são verificados apenas em tempo de execução. Versões mais novas da linguagem estão experimentando com anotações de tipo estáticas opcionais, mas ainda em estágios iniciais de desenvolvimento e adoção.

Existem muitos tipos de dados nativos diferentes no Elixir:

```
1 i = 123
2 f = 3.14
3 s = "héllö" # UTF-8
4 c = 'hello' # Charlists (lista de ints)
5 b = true
6 a = :atom
```

Programa 3.1: *Tipos de Dados em Elixir*

A sintaxe especial `:identificador` é usada para declarar átomos, um conceito do Erlang. Eles são constantes nomeadas por si mesmas e usadas em muitos cenários. Eles são usados para representar códigos de status, nomes de funções, tags ou mesmo representar valores de dados primitivos:

```
1 true == :true
2 false == :false
3 nil == :nil
```

Programa 3.2: *Átomos em Elixir*

Quando se trata de dados compostos, Elixir tem 2 tipos principais (para o escopo deste trabalho): tuplas e listas. As primeiras têm tamanho fixo e são rapidamente indexáveis. As últimas são listas encadeadas simples com comprimento dinâmico. Tuplas são muito usadas para representar dados estruturados simples. Os dados como um todo no Elixir são imutáveis, o que significa que você não pode alterar dados, mas apenas usá-los para produzir novos dados.

```
1 response = {:ok, "Some message"}
2 status = elem(response, 0) # :ok
3 response = put_elem(response, 1, "Another message") # {:ok, "Another message"}
4
5 data = [1, 2, 3, 4]
6 data = ["0" | data] # ["0", 1, 2, 3, 4]
7 data = data ++ [5] # ["0", 1, 2, 3, 4, 5]
```

Programa 3.3: *Estruturas de Dados em Elixir*

O = é chamado de operador de *binding*, isso porque não simplesmente faz atribuição, mas pode ser usado para casamento de padrões.

```

1  x = 1
2  1 = x # Isso funciona
3  2 = x # Isso irá causar um erro
4
5  # Desestruturar tuplas
6  {status, message} = {:err, "the system panicked"}
7
8  IO.puts "It's a #{status}. Because #{message}"
9  # It's a err. Because the system panicked

```

Programa 3.4: *Casamento de Padrões em Elixir*

Mais amplamente, correspondência de padrões pode ser feita com a sintaxe `->`:

```

1  response = {:ok, "Content"}
2
3  case response do
4    {:ok, content} -> IO.puts "Success with content: #{content}"
5    {:err, cause} -> IO.puts "Ugh, it failed due to: #{cause}"
6  end

```

Programa 3.5: *Estruturas de Controle em Elixir*

Em Elixir, existem algumas formas diferentes de declarar funções, e elas podem ser agrupadas com o uso de módulos:

```

1  defmodule Math do
2    # Função pública
3    def add(a, b) do
4      a + b
5    end
6
7    # Forma abreviada de uma linha
8    def mul(a, b), do: a * b
9
10   # Função privada (apenas chamável dentro do módulo)
11   defp square(x), do: x * x
12
13   # Múltiplas declarações de função com casamento de padrões
14   def abs(n) when is_number(n) and n < 0, do: -n
15   def abs(n) when is_number(n), do: n
16 end
17
18 IO.puts(Math.add(2, 3)) # 5
19 IO.puts(Math.mul(4, 5)) # 20

```

Programa 3.6: *Módulos e Funções em Elixir*

3.3.2 Exemplo em Elixir

Agora que você foi introduzido a alguma sintaxe do Elixir, veja um exemplo simples de como implementar um sistema usando atores em Elixir:

```

1  defmodule Echo do
2    def init do
3      # Aguarda por uma mensagem
4      receive do
5        # Obtém o remetente e conteúdo da mensagem
6        {sender, msg} ->
7          # Ecoa a resposta com sucesso (~:ok~)
8          send(sender, {:ok, "Echoing: #{msg}"})
9          # Loop
10         init()
11       end
12     end
13   end
14
15   defmodule Example do
16     def main do
17       # Inicia o ator
18       addr = spawn(Echo, :init, [])
19       # Envia uma mensagem
20       send(addr, {self(), "Hello world"})
21
22       # Aguarda a resposta
23       receive do
24         {:ok, response} ->
25           # Exibe a resposta
26           IO.puts(response)
27       end
28     end
29   end

```

Programa 3.7: *Exemplo de Ator Echo em Elixir*

No exemplo acima, é definido um ator echo simples. Ele recebe uma mensagem que deve conter o endereço do remetente e alguma mensagem. Ele então envia de volta ao remetente uma mensagem com conteúdo `:ok` e a mensagem recebida prefixada com `Echoing:` . Finalmente, há uma chamada recursiva para `init` para que o ator possa lidar com mais mensagens, já que Elixir não tem loops.

No módulo `Example`, a chamada para `spawn` iniciará o ator através da função `init` do módulo `Echo` sem parâmetros. Esta função retornará o PID do processo criado que serve como o endereço do ator. A função `send` envia ao ator recém-criado uma tupla onde o primeiro elemento é o PID do processo atual e o segundo é a string `"Hello world"`.

Neste momento, a chamada para `receive` na função `init` entra em ação. Ela receberá a mensagem e retornará ao remetente uma nova tupla onde o primeiro elemento é `:ok` e o segundo é a string `"Echoing: Hello world"`. Por causa da chamada recursiva, este ator está pronto para receber mais mensagens.

A chamada para `receive` na função `main` agora lidará com a resposta do ator. Ela aguardou pacientemente por uma resposta e pode agora verificar com casamento de padrões que a resposta realmente contém um `:ok` e exibir a string `Echoing: Hello world` no terminal.

Note que esse código não é executado sequencialmente. A chamada de `send` do lado

do módulo `Example` não vai aguardar o ator de `echo` processar a mensagem e gerar uma resposta. Assim que a mensagem é enviada o módulo `Example` fica parado na chamada `receive` aguardando uma resposta.

Capítulo 4

Proposta de Modelos de Atores em Rust

Uma das principais contribuições deste trabalho é a proposta de um *framework* para aplicações centralizadas escritas em Rust usando o Modelo de Atores. Uma vez que a teoria sobre o Modelo de Atores e as implementações conhecidas foram discutidas, é hora de adaptar suas abstrações para as necessidades e características específicas de aplicações centralizadas em Rust.

4.1 Rust

O Modelo de Atores é amplamente conhecido por sua dinamicidade e flexibilidade, com a maioria de suas implementações famosas sendo em runtimes flexíveis como BEAM ou JVM. Rust é bem o oposto disso, uma linguagem moderna com um sistema de tipos forte e um compilador poderoso o suficiente para validar quase tudo em tempo de compilação.

Pode soar contraditório usar uma linguagem rígida com um padrão flexível, mas é exatamente o propósito deste trabalho: experimentar com o Modelo de Atores fora do seu domínio específico. Isso permitirá a experimentação dos limites tanto do Modelo de Atores quanto da linguagem de programação Rust.

Apesar desta distância, Rust ainda possui diversos recursos de linguagem análogos a mecanismos fornecidos por Elixir, fazendo essa adaptação ser possível. Além disso, se Elixir foi eleita a segunda linguagem mais amada nos últimos 3 anos. A mesma pesquisa aponta Rust como sendo a linguagem mais amada desde 2016. Vale ressaltar que Rust 1.0 foi lançado em 2015.

Rust é conhecido por seus mecanismos de segurança que forçam verificação em tempo de compilação, tornando diversos estados inválidos impossíveis de serem representados como código.

Os conceitos mais importantes aqui são:

4.1.1 Segurança de Memória

Um espaço de memória tem um, e apenas um, proprietário (uma variável), este proprietário determina o ciclo de vida do espaço de memória. Uma vez que o proprietário é destruído, o espaço de memória é liberado. O proprietário também determina se a região é mutável ou não.

```

1  fn main() {
2      let a = 10;
3      let mut b = 20;
4
5      // a += 1; // Proibido: a não é mutável
6      b += 1; // Permitido: b é mutável
7
8      // Quando o código termina aqui, a e b são liberados da memória
9  }
```

Programa 4.1: Declaração de variáveis e mutabilidade

```

1  fn main() {
2      let data = [1, 2, 3, 4]; // Cria variável imutável
3      // As duas operações a seguir são proibidas pelo compilador
4      // data = [2, 3, 4, 5];
5      // data[0] = 0;
6
7      let mut data2 = data; // Variável mutável data2
8      // Uso de data proibido pelo compilador desse ponto em diante,
9      // já que seu conteúdo foi transferido para data2
10     // println!("{}", data[0]); // Erro de compilação
11
12     // Permitido
13     data2[0] = 0;
14     data2 = [2, 3, 4, 5];
15 }
```

Programa 4.2: Transferência de propriedade

Um valor pode ser emprestado com o uso de referências, mas existem regras checadas pelo compilador:

1. Referências não podem ser nulas
2. As referências não podem existir por mais tempo do que o dono original
3. Podem existir infinitas referências imutáveis (&) a um valor
4. Pode existir no máximo uma referência mutável (&mut) a um valor (desde que ele seja mutável)
5. Um valor não pode ter ambas referências ao mesmo tempo

```

1  fn main() {
2      let numbers = vec![1, 2, 3, 4];
3      let nref = &numbers;
4      println!("{}", nref[1]); // -> 2
5
6      // Permitido: múltiplas referências imutáveis
```



```

7     let nref2 = &numbers;
8
9     // Proibido: não pode ter referência mutável
10    // quando já existem referências imutáveis
11    // let mref = &mut numbers;
12 }

```

Programa 4.3: *Referências imutáveis*

```

1  fn main() {
2      let mut numbers = vec![1, 2, 3];
3      let nref = &mut numbers;
4
5      // Permitido: operação mutável usando &mut
6      nref.push(4);
7
8      // Proibido: só pode haver uma referência mutável
9      // let nref2 = &mut numbers;
10
11     // Proibido: não pode usar o valor original
12     // enquanto existe referência mutável
13     // numbers.push(5);
14 }

```

Programa 4.4: *Referências mutáveis*

```

1  fn main() {
2      let numbers = vec![1, 2, 3, 4];
3      let nref = &numbers;
4      println!("{}", nref[1]); // -> 2
5
6      drop(numbers); // Libera numbers da memória
7
8      // A partir deste ponto, o compilador proibirá
9      // o uso de tanto numbers quanto nref
10     // println!("{}", nref[1]); // Erro de compilação
11 }

```

Programa 4.5: *Ciclo de vida de referências*

4.1.2 Sistema de Tipos

O sistema de tipos forte e expressivo é uma característica-chave do Rust. Ele não tem herança, mas compensa isso com tuplas, structs, enums e traits.

Tuplas

Uma tupla é simplesmente um tipo produto, tem tamanho fixo e cada campo (nomeado por seu índice) pode ter um tipo diferente. Não é uma coleção!

```

1  fn main() {
2      let value: (i32, f32) = (20, 3.14);
3      println!("{}", value.0); // -> 20
4  }

```

Programa 4.6: *Declaração e uso de tuplas*

Structs

Um struct é similar a uma tupla, mas tem campos nomeados e pode ter métodos com o uso de um bloco `impl`:

```

1  struct Point {
2      timestamp: i32,
3      value: f32
4  }
5
6  impl Point {
7      // Método que pode mutar o valor de self
8      fn add(&mut self, value: f32) {
9          self.value += value;
10     }
11
12     // Método estático que serve como construtor
13     fn new(timestamp: i32, value: f32) -> Self {
14         Self { timestamp, value } // Sintaxe abreviada
15     }
16 }
17
18 fn main() {
19     let mut p = Point::new(20, 2.14);
20     p.add(1.0);
21 }
```

Programa 4.7: *Definição e uso de structs*

Existe também um construto chamado tuple-struct que é um struct mas os campos são nomeados por seus índices, como uma tupla:

```

1  struct Coordinate(i32, i32);
2
3  fn main() {
4      let origin = Coordinate(0, 0);
5  }
```

Programa 4.8: *Tuple-struct*

Enums

Diferente da maioria das linguagens, enums em Rust podem ter payloads já que cada variante pode ser tratada como um struct:

```

1  enum Coordinate {
2      Polar {
3          theta: f32,
4          radius: f32
5      },
6      Cartesian(f32, f32),
7      Origin,
8  }
9
10 fn main() {
11     let origin: Coordinate = Coordinate::Origin;
```

```

12     let point1: Coordinate = Coordinate::Cartesian(1.0, 1.0);
13     let point2: Coordinate = Coordinate::Polar { theta: 45.0, radius: 2.0 };
14 }

```

Programa 4.9: *Enums com payloads*

Traits

Traits são o equivalente Rust de uma interface, define métodos que devem ser implementados por um tipo para que ele tenha uma dada característica. São úteis para limitar tipos genéricos ou sobrecarregar operadores:

```

1  use std::fmt::{Display, Formatter, Error};
2
3  // Display é um trait usado para formatação de string com {}
4  impl Display for Coordinate {
5      fn fmt(&self, f: &mut Formatter<'_>) -> Result<(), Error> {
6          match self {
7              Coordinate::Origin => write!(f, "Origin"),
8              Coordinate::Cartesian(x, y) => write!(f, "({}, {})", x, y),
9              Coordinate::Polar { theta, radius } =>
10                 write!(f, "{} ({})", radius, theta)
11          }
12      }
13  }
14
15  fn main() {
16      let coord = Coordinate::Polar { radius: 10.0, theta: 75.0 };
17      println!("A coordenada é{}", coord); // -> 10 (75)
18  }

```

Programa 4.10: *Implementação de traits*

Com conhecimento básico da linguagem, é momento de começar a introduzir os elementos da proposta em si. Para isso, os elementos da linguagem Elixir que foram discutidos previamente serão usados como referência fortemente.

4.2 Concorrência e Paralelismo

Esta proposta contempla o uso de *green-threads* que não têm suporte nativo em Rust. Isso é porque a linguagem, ao contrário de Elixir, Golang, ou Java, tem um runtime muito mínimo, similar ao runtime do C. O suporte para green-threads é fornecido pela biblioteca Tokio e eles são chamados de *tasks* e serão usados de forma equivalente aos processos Erlang.

As tasks são criadas e executadas com isolamento, seja de forma concorrente ou paralela, dependendo da quantidade de threads do sistema operacional que estão sendo utilizadas pelo ambiente de execução Tokio. Isso permite que múltiplas tasks sejam executadas simultaneamente sem a necessidade de gerenciar manualmente threads do sistema operacional.

```

1  use tokio;
2
3  #[tokio::main]

```

```
4  async fn main() {
5      // Gera a primeira task
6      let task1 = tokio::spawn(async {
7          println!("Task 1 executando!");
8      });
9
10     // Gera a segunda task
11     let task2 = tokio::spawn(async {
12         println!("Task 2 executando!");
13     });
14
15     // Aguarda ambas as tasks terminarem
16     let _ = tokio::join!(task1, task2);
17 }
```

Programa 4.11: *Exemplo de tasks concorrentes com Tokio*

No exemplo acima, duas tasks são criadas usando `tokio::spawn`, cada uma executando de forma independente e concorrente (sem garantia de ordenação). A função `tokio::join!` aguarda que ambas as tasks sejam concluídas antes de prosseguir.

Similarmente ao que foi apresentado nos exemplos com Elixir, onde um ator teria um processo dedicado para lidar com a lógica de processamento de mensagens, aqui será feito uso de *tasks* para isso.

4.3 Compartilhamento de Memória

O isolamento de memória é desejável e uma característica nativa do Modelo de Atores, já que atores são projetados para executar em ambientes fisicamente isolados. Isso aumenta a segurança, mas pode degradar a performance já que o compartilhamento de dados entre máquinas remotas só pode ser feito por cópia.

Mas esse não é o caso para esta proposta, já que a aplicação não é distribuída, temos a garantia que a memória sempre pode ser compartilhada. Além disso, os problemas com compartilhamento de memória em sistemas concorrentes estão geralmente relacionados a condições de corrida e mutabilidade indesejada.

Assim como Elixir garante a imutabilidade dos valores para impedir condições de corrida na comunicação entre processos, Rust e Tokio fornece diversos mecanismos nativos eficientes para compartilhar espaços de memória de forma segura entre *tasks*. Desse modo, usando a abstração correta no cenário correto, é possível de se obter resultados interessantes.

O tipo `Arc<T>` será o principal utilizado para dados complexos, já que permite que um espaço de memória imutável seja compartilhado com contagem de referência atômica (uma vez que ninguém está referenciando-o, ele é liberado). Isso garante que múltiplas tasks possam acessar os mesmos dados de forma segura sem a necessidade de cópia.

4.4 Passagem de Mensagens e Endereços

Tanto Rust quanto Tokio têm uma abstração nativa de passagem de mensagens chamada de canal. Ao contrário da função `send` do Elixir que recebe como parâmetros um endereço e um dado qualquer como mensagem, o método `send` dos canais são fortemente tipados quanto a mensagem e usam um endereço bem definido.

Existem alguns tipos diferentes de canais. Os 2 principais tipos para o escopo deste trabalho são:

4.4.1 *mpsc - Multiple producer, single consumer*

Um canal de múltiplos produtores, consumidor único é a abstração que será usada como endereço de ator. Ele tem 2 extremidades: `Sender` e `Receiver`. O `Sender` pode ser compartilhado entre todos os atores que querem se comunicar com um ator particular apenas criando uma duplicata com o método `clone`. A extremidade `Receiver` é única e será mantida na *task* onde a lógica do ator se encontra e receberá mensagens até o canal ser fechado, ou seja, até não existirem mais clones da extremidade `Sender` correspondente.

Note que o princípio de localidade e segurança do Modelo de Atores nos garante que ninguém deve ser capaz de descobrir o endereço de um ator sem que esse tenha lhe sido dado. Portanto, no momento que não existem mais extremidades `Sender` a um ator, este garantidamente não receberá mais nenhuma mensagem e pode ser finalizado. Isso acontece de forma natural em Rust com o uso de um `loop`, enquanto que em Elixir um ator não recebe nenhum indicativo que seu endereço se tornou desconhecido.

```

1  // Loop que processa mensagens até o canal ser fechado
2  while let Some(message) = rx.recv().await {
3      match message {
4          Message::SetThing { value } => {
5              println!("Definindo valor: {}", value);
6          }
7          Message::GetThing { tx } => {
8              let _ = tx.send(42); // Envia resposta
9          }
10     }
11 }
12 // Quando não há mais Senders, rx.recv() retorna None
13 // e o loop se encerra automaticamente
14 println!("Ator finalizando - canal fechado");

```

Programa 4.12: Exemplo de loop de recebimento de mensagens

No exemplo acima, o loop `while let Some(message) = rx.recv().await` continuará processando mensagens enquanto houver extremidades `Sender` ativas. Quando todas as extremidades `Sender` são descartadas (saem de escopo ou são explicitamente liberadas com o uso de `drop`), o método `recv()` retorna `None`, fazendo com que o loop se encerre naturalmente e a *task* do ator seja finalizada.

Em Elixir, o equivalente seria:

```

1  def loop do

```

```

2      receive do
3          {:set_thing, value} ->
4              IO.puts("Definindo valor: #{value}")
5              loop()
6          {:get_thing, sender} ->
7              send(sender, {:ok, 42})
8              loop()
9      end
10 end

```

Programa 4.13: *Exemplo equivalente em Elixir*

Exemplo acima, em Elixir, é um ator que pode se tornar um ator zumbi, ou seja, que está consumindo recursos ainda que não seja mais usado. Para resolver isso é necessário o uso de um timeout ou de uma mensagem que indique ao ator que ele pode terminar.

É importante notar que o Rust garante, em tempo de compilação, que o receptor de um canal é único. Assim, é garantido que as mensagens chegam ao destino correto. Além disso, o canal tem uma fila com tamanho configurável, e é garantido que a mensagem chegará se a fila não estiver cheia.

4.4.2 oneshot

Um canal que é destruído após uma mensagem ser enviada. Como os canais são unidirecionais, se uma mensagem enviada precisar de uma resposta, um canal oneshot deverá ser criado para envio da resposta.

Também vale ressaltar que o "uso após liberação" de um canal oneshot é verificado em tempo de compilação.

4.5 Representação do Ator

Uma vez definidas quais abstrações a serem usadas, é hora de definir como um ator será em termos de código Rust.

Primeiramente, haverá um módulo para cada ator, e se um ator é subordinado à existência de outro ator pela lógica de negócio, ele pode ser criado como um submódulo.

Neste trabalho, o padrão para módulos será ter um arquivo e uma pasta com o mesmo nome (exceto pela extensão `.rs`). Por exemplo, para um ator chamado App, deve haver um `app.rs` e uma pasta `app/` no mesmo diretório.

Até aqui, três partes principais de um ator foram apresentadas:

- As mensagens que serão enviadas ao ator
- A lógica de processamento das mensagens recebidas
- O endereço que permite a comunicação entre atores

Essas três partes serão traduzidas para partes separadas do código Rust como:

- Um enum privado para definir os tipos das mensagens

- Uma struct privada para gerenciar o estado do ator e definir como lidar com cada mensagem
- Uma struct pública para abstrair o canal *mpsc* que serve como o endereço

Para o exemplo do ator *App*, haverá:

- Um enum *Message* em *app/message.rs* definindo o tipo para as mensagens
- Uma struct *Core* em *app/core.rs* que mantém a lógica do ator
- Uma struct *App* em *app.rs* para servir como a interface pública para dependentes deste ator

Onde *App* deve ser público globalmente, enquanto *Message* e *Core* ficam restritos ao super-módulo *app*.

Note que a convenção é que a struct que serve como interface pública para o ator será nomeada com o nome efetivo do ator para melhor legibilidade. Esta chamada interface pública deve fornecer métodos que abstraem a lógica de construção de mensagens e utilização de canais.

A seguir está destrinchado em mais detalhes o que estará em cada uma das 3 partes.

4.5.1 Mensagens

Assim como tudo em Rust, as mensagens enviadas através de canais devem ser tipadas. Caso um ator só execute um tipo de ação, ou seja, só receba um tipo de mensagem então qualquer tipo pode ser usado para representar as mensagens. Caso ele lide com diferentes tipos de mensagens (o que acontece na maioria dos casos) um *enum* será usado.

No contexto do exemplo dado, o *enum* a ser criado no arquivo *app/message.rs* seria:

```
1  enum Message {
2      SetThing { value: i32 },
3      GetThing { tx: oneshot::Sender<i32> }
4  }
```

Programa 4.14: Exemplo de mensagens tipadas

A maior vantagem de tipar mensagens com *enums* é que o envio e o tratamento são verificados em tempo de compilação. Isso cria uma garantia forte sobre quais mensagens que podem estar envolvidas em uma dada comunicação e o compilador Rust garantirá que:

- O receptor fornecerá tratamento explícito para todas as mensagens possíveis
- O remetente nunca enviará uma mensagem inválida

As garantias acima não se aplicam à implementação do Modelo de Atores em Elixir.

4.5.2 Lógica

Toda a lógica de processamento de mensagens para um ator ficará dentro de uma *struct* chamada de *Core*. Ela será também onde será feito o gerenciamento de estado do ator e pelos detalhes específicos para criação da *task* e do canal *mpsc* dedicado para a comunicação.

No contexto do exemplo dado, a *struct* a ser criada no arquivo `app/core.rs` seria:

```
1  pub struct Core {
2      thing: i32,
3  }
```

Programa 4.15: *Exemplo de struct Core para lógica do ator*

O Core deve ser instanciado com a invocação do seu construtor, nesse momento as dependências do ator deverão ser injetadas como argumentos do construtor. O construtor será chamado `new` caso não possa falhar, caso contrário será chamado de `build`. Por exemplo, a inicialização de um ator pode depender do valor de uma variável de ambiente que talvez não exista, e isso leva a uma falha.

```
1  impl Core {
2      pub fn new(thing: i32) -> Self {
3          Self {
4              thing
5          }
6      }
7  }
```

Programa 4.16: *Exemplo de construtor*

Deverá ser fornecido um método de instância `init` responsável por conter a lógica de processamento das mensagens recebidas, delegando elas para os métodos responsáveis.

```
1  impl Core {
2      // ...
3      pub async fn init(mut self, mut rx: mpsc::Receiver<Message>) {
4          while let Some(message) = rx.recv().await {
5              match message {
6                  Message::SetThing { value } => {
7                      self.handle_set_thing(value);
8                  }
9                  Message::GetThing { tx } => {
10                     let result = self.handle_get_thing();
11                     let _ = tx.send(result);
12                 }
13             }
14         }
15     }
16
17     fn handle_set_thing(&mut self, value: i32) {
18         self.thing = value;
19     }
20
21     fn handle_get_thing(&self) -> i32 {
22         self.thing
23     }
24 }
```

Programa 4.17: *Exemplo de construtor*

Note que o método `init` recebe `self` como primeiro argumento, isso significa que ele consome a instância. Em outras palavras, aquele que tiver posse de uma instância do

Core do App a perderá assim que `init` for invocada, impedindo que um mesmo objeto seja instanciado como 2 atores compartilhando o mesmo espaço de memória. O segundo parâmetro é a ponta de recebimento de mensagens do canal para se comunicar com o ator.

Adicionalmente, cada ramo do bloco `match` apenas obtém os valores da mensagem e delega a execução para um método privado `handle_`. Em caso de necessidade de enviar resposta, isso não deve ser feito dentro método `handle`, assegurando assim o princípio de responsabilidade única.

4.5.3 Endereço

Para se aproveitar ao máximo dos recursos que a linguagem Rust oferece, é ideal a criação de um tipo que abstraia a lógica de uso dos canais, tanto para facilitar a leitura do código, quanto para diminuir duplicação de código. Ao contrário do que é feito em Elixir, onde se atribui o nome do ator ao módulo onde se encontra a lógica de processamento, aqui o nome será atribuído a essa abstração sobre o endereço.

```

1  use tokio::sync::{mpsc, oneshot};
2  use crate::error::{AppError, FatalActorError};
3
4  pub struct App {
5      tx: mpsc::Sender<Message>,
6  }
7
8  impl App {
9      pub fn spawn(value: i32) -> Self {
10         let (tx, rx) = mpsc::channel(crate::BUFFER_SIZE);
11         let core = Core::new(value);
12         let _ = tokio::spawn(async move {
13             core.init(rx).await;
14         });
15
16         Self { tx }
17     }
18
19     pub async fn thing(&self) -> Result<i32, AppError> {
20         let (tx, rx) = oneshot::channel();
21         let message = Message::GetThing { tx };
22
23         self.tx.send(message).await
24             .map_err(|_| {
25                 AppError::Fatal(FatalActorError::ActorSendFailed {
26                     actor_name: "App",
27                     operation: "get thing".to_string(),
28                 })
29             })?;
30
31         rx.await
32             .map_err(|_| {
33                 AppError::Fatal(FatalActorError::ActorRecvFailed {
34                     actor_name: "App",
35                     operation: "get thing".to_string(),
36                 })
37             })?

```

```

38     }
39
40     pub async fn thing_alt(&self) -> Result<oneshot::Receiver<Result<i32,
        AppError>>, AppError> {
41         let (tx, rx) = oneshot::channel();
42         let message = Message::GetThing { tx };
43
44         self.tx.send(message).await
45             .map_err(|_| {
46                 AppError::Fatal(FatalActorError::ActorSendFailed {
47                     actor_name: "App",
48                     operation: "get thing".to_string(),
49                 })
50             })?;
51
52         Ok(rx)
53     }
54
55     pub async fn set_thing(&self, value: i32) -> Result<(), AppError> {
56         let message = Message::SetThing { value };
57
58         self.tx.send(message).await
59             .map_err(|_| {
60                 AppError::Fatal(FatalActorError::ActorSendFailed {
61                     actor_name: "App",
62                     operation: "set thing".to_string(),
63                 })
64             })
65     }
66 }

```

Programa 4.18: Exemplo de struct para endereço do ator

No exemplo acima, a struct `App` encapsula o `Sender` do canal `mpsc` e fornece métodos públicos que abstraem completamente a lógica de envio de mensagens. O método `thing()` cria um canal `oneshot` para receber a resposta, envia a mensagem `GetThing` e aguarda a resposta. O método `thing_alt()` oferece uma alternativa onde retorna o `Receiver` diretamente, permitindo que o invocador decida quando aguardar pela resposta. O método `set_thing()` simplesmente envia a mensagem `SetThing` com o valor fornecido.

Essa abstração permite que outros atores interajam com o ator sem precisar se preocupar com detalhes de comunicação por canais, mantendo a interface limpa e tipada.

4.6 Tratamento de Erros

Um aspecto fundamental do modelo de atores proposto é o tratamento de erros de forma consistente e tipada. Diferente de linguagens como `Elixir`, onde erros são tratados principalmente através de padrões de `matching` com tuplas `{:ok, value}` ou `{:error, reason}`, `Rust` oferece o tipo `Result<T, E>` que força, em tempo de compilação, o tratamento explícito de erros.

Para facilitar a definição e manipulação de erros, este trabalho utiliza duas bibliotecas complementares: `thiserror`, que permite criar hierarquias de tipos de erro usando `derive`

macros, e `miette`, que fornece diagnósticos ricos e informativos para erros, facilitando o debugging e a experiência do desenvolvedor.

4.6.1 Hierarquia de Erros

O sistema de erros é organizado hierarquicamente, com três níveis principais:

- **Erros Fatais:** Falhas de nível de sistema que indicam que um ator morreu ou que a comunicação falhou de forma irreversível
- **Erros Específicos de Ator:** Cada ator define apenas os erros que ele pode emitir, garantindo uma interface de erro bem definida

4.6.2 Erros Fatais

Erros fatais representam falhas de comunicação que não podem ser recuperadas sem reiniciar o componente afetado. Esses erros são comuns a todos os atores e indicam problemas com o sistema de passagem de mensagens:

```

1  use miette::Diagnostic;
2  use thiserror::Error;
3
4  #[derive(Debug, Error, Diagnostic)]
5  #[error("Fatal actor error")]
6  pub enum FatalActorError {
7      /// Falha ao enviar mensagem para o ator
8      #[error("Failed to send message to actor '{actor_name}' for operation '{operation}'")]
9      #[diagnostic(
10         code(fatal::actor_send_failed),
11         help("The message could not be sent to the actor. The actor may have died or the channel may be closed.")
12     )]
13     ActorSendFailed {
14         actor_name: &'static str,
15         operation: String,
16     },
17
18     /// Falha ao receber resposta do ator
19     #[error("Failed to receive response from actor '{actor_name}' for operation '{operation}'")]
20     #[diagnostic(
21         code(fatal::actor_recv_failed),
22         help("The response could not be received from the actor. The actor may have died or the channel may be closed.")
23     )]
24     ActorRecvFailed {
25         actor_name: &'static str,
26         operation: String,
27         #[source]
28         source: tokio::sync::oneshot::error::RecvError,
29     },
30 }

```

Programa 4.19: Definição de erros fatais

Erros fatais ocorrem quando o canal de comunicação é fechado, o que acontece quando não há mais extremidades Sender ativas para o ator. Isso pode indicar que o ator terminou inesperadamente ou que todas as referências a ele foram liberadas.

4.6.3 Erros Específicos de Ator

Cada ator define seu próprio tipo de erro, que encapsula tanto os erros específicos do domínio quanto os erros fatais de comunicação. Por exemplo, o ator (Env) define:

```

1  #[derive(Debug, Error, Diagnostic)]
2  pub enum EnvError {
3      /// Variável de ambiente não encontrada
4      NotFound {
5          name: String,
6      },
7
8      /// Erro fatal ocorrido durante operações do ator
9      #[error(transparent)]
10     #[diagnostic(transparent)]
11     Fatal(#[from] FatalActorError),
12 }

```

Programa 4.20: Exemplo de erro específico de ator

A variante `Fatal` permite que erros fatais sejam propagados automaticamente através do operador `?`, mantendo a hierarquia de erros enquanto permite tratamento específico quando necessário.

4.6.4 Propagação de Erros através de Canais

Quando um ator precisa retornar um resultado que pode falhar, o tipo de retorno deve ser `Result<T, E>`, onde `E` é o tipo de erro específico do ator. O resultado é enviado através do canal oneshot como parte da resposta:

```

1  impl Core {
2      pub async fn init(self, mut rx: mpsc::Receiver<Message>) {
3          while let Some(msg) = rx.recv().await {
4              match msg {
5                  Message::Get { tx, key } => {
6                      // Processa a operação e pode retornar um erro
7                      let result = self.handle_get_env(key);
8                      let _ = tx.send(result);
9                  }
10                 // ...
11             }
12         }
13     }
14
15     fn handle_get_env(&self, key: ArcOsStr) -> Result<ArcStr, EnvError> {
16         std::env::var(key.as_ref())
17             .map(|s| ArcStr::from(&s))
18             .map_err(|_| EnvError::NotFound {
19                 name: key.to_string_lossy().to_string(),
20             })

```

```

21     }
22 }

```

Programa 4.21: *Processamento de mensagens com tratamento de erros*

4.6.5 Tratamento de Erros na Interface Pública

Na interface pública do ator, erros podem ocorrer em dois pontos:

1. Ao enviar a mensagem para o ator (falha de comunicação)
2. Ao receber a resposta do ator (falha de comunicação ou erro de domínio)

Ambos os casos devem ser tratados e convertidos em erros fatais quando apropriado:

```

1  impl Env {
2      pub async fn env(&self, key: ArcOsStr) -> Result<ArcStr, EnvError> {
3          let (tx, rx) = tokio::sync::oneshot::channel();
4
5          // Erro ao enviar mensagem -> Erro fatal
6          self.tx
7              .send(Message::Get { tx, key })
8              .await
9              .map_err(|_e| {
10                  EnvError::Fatal(FatalActorError::ActorSendFailed {
11                      actor_name: "Env",
12                      operation: "get environment variable".to_string(),
13                  })
14              })?;
15
16          // Erro ao receber resposta -> Erro fatal ou erro de domínio
17          rx.await
18              .map_err(|e| {
19                  EnvError::Fatal(FatalActorError::ActorRecvFailed {
20                      actor_name: "Env",
21                      operation: "get environment variable".to_string(),
22                      source: e,
23                  })
24              })?
25      }
26 }

```

Programa 4.22: *Tratamento de erros na interface pública do ator*

Note que o operador ? é usado para propagar erros automaticamente. Se o envio da mensagem falhar (por exemplo, porque o canal foi fechado), o erro é convertido em um `FatalActorError`. Se a recepção falhar, também é convertido em um erro fatal. Caso contrário, o resultado (que pode conter um erro de domínio como `NotFound`) é retornado.

4.6.6 Erros Recuperáveis e Não-Recuperáveis

O sistema distingue entre erros recuperáveis e não-recuperáveis através de um atributo `retryable` que indica se a operação pode ser tentada novamente.

Essa distinção permite que a aplicação tome decisões inteligentes sobre como lidar com erros, como implementar retry automático para erros, ou desligar graciosamente para erros fatais.

4.6.7 Funções Auxiliares para Criação de Erros

Para simplificar a criação de erros específicos de domínio a partir de erros de bibliotecas externas, o sistema fornece funções auxiliares. Por exemplo, para converter erros de IO do sistema de arquivos em `FsError`:

```

1  pub fn fs_error(
2      path: Option<impl Into<String>>,
3      operation: impl Into<String>,
4      source: std::io::Error,
5  ) -> FsError {
6      FsError::OperationFailed {
7          path: path.map(|p| p.into()),
8          operation: operation.into(),
9          retryable: source.is_retryable(),
10         source,
11     }
12 }
```

Programa 4.23: Função auxiliar para criação de erros de sistema de arquivos

Essas funções auxiliares garantem que informações relevantes sobre o contexto do erro sejam preservadas e que a informação sobre recuperabilidade seja determinada automaticamente baseada nas características do erro subjacente.

4.7 Testabilidade e Mocks

Uma das principais vantagens da arquitetura baseada em atores é a facilidade de testar componentes isoladamente. Como cada ator possui dependências injetadas através de suas interfaces, é possível substituir dependências reais por implementações mockadas durante os testes.

4.7.1 Mockall

Para simplificar a criação de mocks, este trabalho faz uso da biblioteca `mockall`, que gera automaticamente implementações mock a partir de traits ou structs. Isso elimina a necessidade de escrever manualmente código boilerplate para mocks e garante que os mocks permaneçam sincronizados com as interfaces reais. O código para mock de um ator deve ser colocado no arquivo `mock.rs` do módulo do ator e o módulo deve ser declarado como:

```

1  #[cfg(test)]
2  pub mod mock;
```

Programa 4.24: Exemplo de declaração de módulo para mock

Desse modo, o código do mock não será incluso no binário de produção.

```

1  mock!{
2      #[derive(Debug)]
3      pub App {
4          pub async fn operation(&self, params: Params) -> Result<Value>;
5      }
6
7      impl Clone for Actor {
8          fn clone(&self) -> Self;
9      }
10 }

```

Programa 4.25: *Exemplo de mock usando mockall*

A macro `mock!` do `mockall` gera automaticamente uma estrutura com nome `MockApp` que implementa todos os métodos especificados e permite configurar comportamentos esperados para cada invocação. Além disso, a macro gera implementações de traits como `Debug` e `Clone`, mantendo a compatibilidade com a interface original.

4.7.2 Integração de Mocks

Para permitir a substituição fácil entre implementações reais e mockadas, o sistema usa atributos de compilação condicional (`#[cfg(test)]` e `#[cfg(not(test))]`) para alternar entre as implementações:

```

1  #[cfg(not(test))]
2  use crate::other_actor::OtherActor;
3
4  #[cfg(test)]
5  use crate::other_actor::mock::MockOtherActor as OtherActor;
6
7  pub struct App {
8      dependency: OtherActor,
9  }
10
11 impl App {
12     pub fn new(dependency: OtherActor) -> Self {
13         Self { dependency }
14     }
15 }

```

Programa 4.26: *Alternância entre implementações reais e mockadas*

Durante a compilação normal, o sistema usa a implementação real `OtherActor`. Durante a compilação de testes, automaticamente substitui por `MockOtherActor`. Com o uso de compilação condicional, é possível implementar uma infraestrutura de testes bastante robusta que terá nenhum impacto no código de produção.

4.7.3 Testes Unitários

Com mocks em vigor, escrever testes unitários torna-se direto. Cada teste pode configurar o comportamento esperado de suas dependências mockadas e verificar que o ator sob teste se comporta corretamente:

```
1  #[tokio::test]
2  async fn test_my_actor_operation() {
3      let mut mock_dep = MockOtherActor::new();
4      mock_dep
5          .expect_get_value()
6          .times(1)
7          .returning(|| Ok(42));
8
9      let actor = App::spawn(mock_dep);
10     let result = actor.operation().await;
11
12     assert_eq!(result, Ok(42));
13 }
```

Programa 4.27: *Exemplo de teste unitário com mocks*

Os mocks permitem testar tanto cenários de sucesso quanto de falha, validando o tratamento de erros e garantindo que as interações entre atores funcionam corretamente em isolamento.

Capítulo 5

Estudo de Caso: Patch-Hub

Uma vez apresentadas as propostas de adaptação do Modelo de Atores para experimentação fora do domínio dos sistemas distribuídos, chega o momento de realizar um estudo de caso para aplicar a proposta em um projeto real para posterior análise de resultados. O projeto escolhido foi uma ferramenta de código aberto desenvolvida sob a organização KWorkflow (kw) chamado de Patch-Hub.

5.1 Contexto

Para compreender adequadamente o projeto Patch-Hub e sua importância, é necessário entender o ecossistema de desenvolvimento do kernel Linux e os desafios enfrentados pelos desenvolvedores. O Linux é um sistema operacional de código aberto baseado no kernel Unix-like desenvolvido por Linus Torvalds em 1991 [WIKIPEDIA, 2024a](#). O kernel Linux é o núcleo do sistema operacional, responsável por gerenciar recursos de hardware, processos e comunicação entre componentes do sistema [WIKIPEDIA, 2024b](#).

5.1.1 O Processo de Contribuição para o Kernel Linux

O desenvolvimento do kernel Linux é um processo colaborativo que envolve milhares de desenvolvedores ao redor do mundo. Devido à sua complexidade e tamanho, o kernel é organizado em subsistemas especializados, cada um com responsabilidades específicas [LINUX KERNEL DOCUMENTATION, 2024b](#). Contribuições para o kernel, conhecidas como *patches*, são tradicionalmente enviadas através de listas de discussão públicas por email e revisadas por mantenedores especializados [LINUX KERNEL DOCUMENTATION, 2024a](#).

Este processo tradicional de contribuição apresenta diversos desafios. Os desenvolvedores precisam se inscrever em múltiplas listas de email e gerenciar um volume considerável de mensagens. Isso dificulta muito acompanhar o status de suas contribuições.

5.1.2 O Projeto `lore.kernel.org`

Para mitigar essas dificuldades, foi criado o projeto `lore.kernel.org`, um arquivo público que fornece uma interface web para visualizar e pesquisar mensagens das listas de dis-

cussão do kernel [TADOKORO, 2023](#). O [lore.kernel.org](#) oferece uma API que permite acesso programático aos patches arquivados, incluindo funcionalidades de busca, paginação e filtros avançados.

A API do lore suporta consultas complexas através de parâmetros de query string, permitindo filtrar patches por critérios como assunto, autor, data e conteúdo. Esta funcionalidade é fundamental para ferramentas que buscam automatizar ou simplificar a interação com o ecossistema de patches do kernel.

5.1.3 O Projeto KWorkflow

O projeto KWorkflow (kw) surgiu como uma resposta aos desafios de configuração e complexidade do ambiente de desenvolvimento para o kernel Linux [KWORKFLOW, 2024a](#). O kw é descrito como uma "ferramenta de fluxo de trabalho de desenvolvedor de kernel" que visa reduzir a sobrecarga de configuração e fornecer ferramentas para tarefas comuns do desenvolvimento de kernel.

O projeto oferece um utilitário de terminal unificado para diversas operações como compilação, deploy, debug e gerenciamento de configurações, consolidando ferramentas anteriormente dispersas.

5.1.4 O Patch-Hub

O Patch-Hub é uma ferramenta desenvolvida como parte do ecossistema KWorkflow, especificamente projetada para interagir com a API do [lore.kernel.org](#) [KWORKFLOW, 2024b](#). Sua missão é agilizar a navegação e revisão de contribuições para o kernel Linux, fornecendo uma interface de usuário baseada em terminal (TUI) que simplifica o processo de descoberta, análise e acompanhamento de patches.

A ferramenta permite aos desenvolvedores navegar por listas de patches, aplicar filtros de busca, visualizar conteúdo de patches e gerenciar seu fluxo de trabalho de revisão diretamente do terminal, eliminando a necessidade de alternar entre interfaces web e ferramentas de linha de comando.

5.2 Arquitetura Original

A implementação original do Patch-Hub seguia uma arquitetura majoritariamente single-threaded e monolítica com inspirações no padrão arquitetural MVC. A organização do código era dada em termos dos seguintes módulos:

- **lore**: Interface para interagir com a API do lore para buscar dados de patches
- **ui**: Renderização de terminal e lógica de exibição
- **cli**: Parsing de argumentos de linha de comando
- **app**: Estado central da aplicação e gerenciamento de dados
 - **logger**: Gerencia funcionalidade de logging

- popup: Cria e gerencia diálogos popup
- config: Gerencia configurações da aplicação
- **handler**: Manipulação de entrada do usuário e processamento de eventos

5.3 Funcionalidades Principais

A implementação do Patch-Hub (tanto a original quanto a refatorada) fornece um conjunto de funcionalidades para interagir com o arquivo de patches do kernel Linux. O sistema permite a seleção e navegação de listas de discussão arquivadas no *lore.kernel.org*, apresentando os conjuntos de patches associados. Ao examinar uma série específica, a interface disponibiliza o conteúdo individual dos patches juntamente com metadados relevantes, incluindo título, autor, versão, quantidade de patches e timestamp da última atualização.

– imagem –

O sistema implementa operações fundamentais para o workflow de revisão de patches: aplicação de patches em repositórios locais do kernel, organização de séries em um sistema de favoritos, e composição de respostas com tags de revisão padrão, como Reviewed-by.

A renderização do conteúdo dos patches é implementada com suporte a múltiplos backends, incluindo ferramentas externas, além de um renderizador padrão. Esta arquitetura permite variação na apresentação visual dos patches ao gosto do usuário sem comprometer a portabilidade da aplicação.

5.4 Limitações da Arquitetura Original

Apesar de sua funcionalidade, a implementação original do Patch-Hub apresentava limitações arquiteturais fundamentais que comprometiam sua extensibilidade, manutenibilidade e capacidade de evolução. Essas limitações vinham de decisões de design que, embora funcionais para o escopo inicial, criavam gargalos estruturais que dificultavam a adição de novas funcionalidades e a manutenção do código no longo prazo.

5.4.1 Gargalo de Mutação Centralizada

O principal problema arquitetural reside na concentração de todo o estado da aplicação em uma única estrutura `App` que deve ser passada como `&mut App` através de todo o código. Esta abordagem cria um *gargalo de mutação centralizada* que viola princípios fundamentais de design modular.

O sistema de ownership do Rust impõe que quando uma função recebe `&mut App`, ela adquire uma referência mutável exclusiva de toda a estrutura. Isso significa que:

- Nenhum outro código pode ler ou escrever qualquer campo de `App` até que essa referência seja liberada

- Campos não podem ser emprestados independentemente enquanto `&mut App` está ativa
- Componentes internos não podem modificar componentes irmãos sem liberar a referência primeiro

Esta restrição força a centralização de toda lógica de mutação nos métodos de `App`, impedindo que componentes gerenciem seu próprio estado de forma independente.

O resultado é uma violação do princípio da responsabilidade única, forçando a estrutura central a conhecer detalhes de implementação de todos os seus componentes. Além disso, o princípio da inversão de dependência também é isolado fazendo com que a estrutura de alto nível dependa diretamente de mecanismos baixo nível de seus componentes internos.

5.4.2 Impossibilidade de Composição Modular

O design atual impede a composição modular de handlers e componentes devido às restrições de borrowing. Handlers de eventos devem receber `&mut App` inteiro mesmo quando precisam acessar apenas um subconjunto específico de dados. Isso cria uma situação onde handlers não podem ser decompostos em funções auxiliares menores e também funções auxiliares que precisam de múltiplos campos de `App` enfrentam conflitos por tentarem pegar referências a partes de algo que já está possui uma referência mutável.

Esta limitação força a criação de métodos monolíticos em `App` que conhecem detalhes de implementação de múltiplos componentes, aumentando a complexidade e reduzindo a testabilidade.

5.4.3 Atualizações de Estado Intercomponentes Problemáticas

A arquitetura torna impossível a atualização direta de estado entre componentes, forçando todas as interações através da estrutura central `App`. Quando um componente precisa modificar o estado de outro componente irmão, ele deve passar pela estrutura `App` como intermediário, criando acoplamento forte entre componentes que deveriam ser independentes.

5.4.4 Concorrência e Responsividade Limitadas

A natureza single-threaded da aplicação cria gargalos significativos durante operações de I/O, resultando em:

- Interfaces não responsivas durante requisições de rede
- Bloqueio da UI durante operações de sistema de arquivos
- Lógica complexa para implementar telas de carregamento
- Impossibilidade de realizar operações paralelas independentes

Para contornar esse problema, foi feito uso de mecanismos complicados para deixar certas regiões do código paralelas. A complicação aqui é que referências em Rust não podem ser compartilhadas entre threads pois não há garantias que uma thread vá ser

encerrada antes do dono do valor referido, o que poderia causar vazamentos de memória ou condições de corrida indesejáveis.

5.4.5 Dificuldades de Testabilidade

A arquitetura monolítica torna extremamente difícil a criação de testes unitários eficazes. Testes de handlers individuais requerem a construção de toda a estrutura App, incluindo dependências que podem não ser relevantes para o teste específico. Isso resulta em:

- Testes lentos devido à necessidade de inicialização completa
- Testes frágeis que quebram quando componentes não relacionados são modificados
- Impossibilidade de isolar comportamento específico para teste
- Dificuldade em criar mocks e stubs para dependências externas

Esta limitação compromete significativamente a capacidade de garantir qualidade através de testes automatizados, aumentando o risco de regressões em mudanças futuras.

Nota: As limitações descritas acima pertenciam à implementação monolítica original do Patch-Hub. A refatoração para o Modelo de Atores resolve essas questões através de message passing, gestão independente de estado por ator, e integração com mockall para testabilidade.

5.5 Objetivos

Fica evidente que o Patch-Hub necessita de refatorações para tentar resolver os problemas supracitados. Em especial, fica notável uma grande necessidade de melhorias do ponto de vista da flexibilidade do sistema. Tendo em vista que o Modelo de Atores proporciona alta modularidade e baixo acoplamento aos sistemas, isso torna o Patch-Hub um candidato interessante para experimentação e avaliação de valor da proposta. Além disso, o Patch-Hub toca em domínios suficientemente diversos para que o experimento seja bastante completo.

Este trabalho não criará uma reescrita completa com 100% de compatibilidade de funcionalidades. Ao invés disso, um subconjunto das funcionalidades foi escolhido para implementação, suficiente para ter as funcionalidades principais do Patch-Hub, que são:

- Uma interface de usuário de texto interativa
- Integração com API do Lore com suporte a cache
- Navegar através das listas de discussão catalogadas
- Acesso a um feed de patches enviados para uma lista de discussão
- Fornecer uma visualização do conteúdo e metadados de um patch
- Configurações personalizáveis
- Um sistema de log

5.6 Modelagem com Atores

Com objetivos estabelecidos, resta apenas modelar a arquitetura do Patch-Hub com o uso de atores. A modelagem é feita adotando os princípios SOLID como guias. Os atores serão aqui categorizados em 2 grupos considerando o tipo de interação que eles são responsáveis: externas ou internas.

Os atores responsáveis por interações com componentes externos ao sistema, como por exemplo o sistema de arquivos, a internet ou o terminal. Este grupo será chamado de atores de integração pois permitem a integração do programa com outros sistemas, geral assim efeitos no mundo real.

O segundo grupo é formado pelos atores que irão interagir apenas com demais atores, ou seja, componentes internos ao sistema. Este grupo tem uma diferença fundamental no que diz respeito a sua testabilidade. Por dependerem apenas de outros atores, testes unitários se tornam triviais uma vez que se criem dublês (mocks). Uma vez que, testando um ator contra dublês, dá a garantia que apenas o código do próprio ator é o que influencia no resultado dos testes. Este grupo será chamado de atores de lógica pois conterá, majoritariamente, atores modelados pela lógica de negócio da aplicação.

5.6.1 Atores de Integração

Primeiramente é necessário identificar quais são as capacidades do Patch-Hub que dependem unicamente de interação com sistemas externos ao programa, elas serão:

- Operações do sistema de arquivos
- Requisições HTTP
- Manipulação de variáveis de ambiente
- Controle de entrada e saída do terminal
- Execução de comandos externos

Essas capacidades não fazem parte da lógica de negócio do Patch-Hub, mas servem de bases para permitir a sua implementação. Cada uma dessas capacidades pode ser traduzida para um ator em nossa aplicação seguindo os princípios SOLID, especialmente os princípios de responsabilidade única e de segregação de interfaces.

Sistema de Arquivos

O ator Sistema de Arquivos é responsável por gerenciar as operações de leitura, escrita, remoção e listagem de arquivos e diretórios. Ele é responsável por garantir que as operações sejam realizadas de forma segura, garantindo que o sistema de arquivos seja consistente e que os arquivos e diretórios sejam gerenciados corretamente.

Mensagens:

- `ReadFile`: Abre um arquivo apenas para leitura (não cria se não existir)
- `WriteFile`: Abre um arquivo para escrita (trunca conteúdo, cria se necessário)

- `AppendFile`: Abre um arquivo para anexar (cria se necessário)
- `RemoveFile`: Remove um arquivo do sistema de arquivos
- `ReadDir`: Lê o conteúdo de um diretório
- `MkDir`: Cria um diretório e seus pais
- `Rmdir`: Remove um diretório e seu conteúdo

Rede

O ator Rede é responsável por gerenciar as operações de requisição HTTP. Ele é o mecanismo base que vai permitir a integração do sistema com a API web do `lore.kernel.org`.

Depende:

- **Configurações**: Para obter configurações de timeout e outras configurações de rede
- **Log**: Para registrar operações de rede para posterior análise

Mensagens:

- `Get`: Executa uma requisição HTTP GET para a URL especificada com headers opcionais
- `Post`: Executa uma requisição HTTP POST para a URL especificada com headers opcionais e corpo
- `Put`: Executa uma requisição HTTP PUT para a URL especificada com headers opcionais e corpo
- `Delete`: Executa uma requisição HTTP DELETE para a URL especificada com headers opcionais e corpo
- `Patch`: Executa uma requisição HTTP PATCH para a URL especificada com headers opcionais e corpo

Variáveis de Ambiente

O ator Variáveis de Ambiente é responsável por gerenciar as operações de definição, remoção e recuperação de variáveis de ambiente.

Mensagens:

- `Set`: Define uma variável de ambiente para um valor especificado
- `Unset`: Remove uma variável de ambiente
- `Get`: Recupera o valor de uma variável de ambiente (retorna `VarError` se não encontrada)

Terminal

O ator Terminal gerencia o loop de eventos da interface de usuário baseada na biblioteca `Cursive`. Ele mantém uma fila FIFO interna de eventos de UI (teclas pressionadas, seleções,

etc.) além de ser responsável por desenhar no terminal. Ele também expõe uma API baseada em mensagens para atualizar a interface e recuperar os eventos de UI ocorridos.

Depende:

- **Log:** Para registrar eventos do terminal para posterior análise

Mensagens:

- **Show:** Atualiza a tela exibida com um novo tipo de tela
- **GetUiEvent:** Recupera o próximo evento de UI da fila
- **ClearUiEvents:** Limpa a fila de eventos de UI
- **Quit:** Termina a UI e sai da aplicação

O método `spawn()` do ator `Terminal` retorna uma tupla contendo a interface do ator e um `JoinHandle` que permite que outros atores detectem quando o terminal foi encerrado, permitindo encerramento gracioso da aplicação. Este design evita a necessidade de envolver o `JoinHandle` em um `Arc` e simplifica o gerenciamento do ciclo de vida do terminal.

Shell

O ator `Shell` é responsável por executar comandos integrando o Patch-Hub ao shell do sistema hospedeiro. Através dele é possível expandir as funcionalidades do Patch-Hub com o uso de ferramentas externas.

Depende:

- **Log:** Para registrar operações de execução de comandos para posterior análise

Mensagens:

- **Execute:** Executa um programa externo com comando, argumentos e entrada `stdin` opcionais retornando a saída padrão do programa

5.6.2 Atores de Lógica

É chegado o momento de se identificar as demais capacidades relacionadas à lógica da aplicação que não foram cobertas pelos atores de integração. São elas:

- Registrar eventos com um sistema de logs
- Gerenciar configurações
- Acessar a Lore API
- Suporte a caching das requests feitas
- Desenhar a interface do usuário

Logs

Fornece um serviço de logging centralizado e thread-safe. Ele recebe mensagens de log de outros atores e as escreve em um arquivo de log designado, fornecendo diferentes níveis de log para debugging e monitoramento.

Depende:

- **Sistema de Arquivos:** Para persistir logs em um arquivo

Mensagens:

- **Log:** Registra uma mensagem com um nível de log específico (Info, Warning, Error) através de uma estrutura `LogMessage`
- **Flush:** Escreve as mensagens de log em buffer para `stderr` e encerra o logger
- **CollectGarbage:** Executa a limpeza de arquivos de log antigos baseado na idade máxima configurada

Configurações

Responsável por carregar configurações de um arquivo de configuração, fornecê-las a outros atores sob demanda e persistir mudanças.

Depende:

- **Sistema de Arquivos:** Para salvar e carregar o arquivo de configuração

Mensagens:

- **Load:** Recarrega a configuração do arquivo fonte
- **Save:** Salva a configuração atual no arquivo fonte
- **GetPath:** Recupera o valor de uma configuração baseada em caminho (ex: `LogDir`, `CachePath`)
- **SetPath:** Atualiza o valor de uma configuração baseada em caminho
- **GetLogLevel:** Recupera o nível de log atual
- **SetLogLevel:** Atualiza o nível de log
- **GetUSize:** Recupera o valor de uma configuração numérica (ex: `MaxAge`, `Timeout`)
- **SetUSize:** Atualiza o valor de uma configuração numérica
- **GetRenderer:** Recupera a configuração de um renderizador (ex: `PatchRenderer`)
- **SetRenderer:** Atualiza a configuração de um renderizador

Lore API

Gerencia toda a comunicação com a API externa do Lore patchwork. Ele abstrai os detalhes das requisições HTTP e parsing de dados, fornecendo uma interface limpa e tipada para buscar dados como listas de discussão, séries de patches e patches individuais.

Ele atua como um tradutor entre as estruturas de dados internas da aplicação e as respostas brutas da API. Ele não realiza nenhum cache por si só.

Depende:

- **Rede:** Para executar as requisições HTTP subjacentes

Mensagens:

- `GetAvailableLists`: Busca todas as listas de discussão disponíveis (sem paginação)
- `GetAvailableListsPage`: Busca uma lista paginada de listas de discussão disponíveis
- `GetPatchFeedPage`: Busca um feed paginado de patches para uma lista de discussão específica
- `GetRawPatch`: Busca o conteúdo bruto de um único patch em formato texto
- `GetPatchHtml`: Busca o conteúdo HTML de um único patch
- `GetPatchMetadata`: Busca os metadados de um único patch em formato JSON

Interface de Usuário

O ator UI gerencia a lógica de navegação e estado da interface, coordenando a comunicação entre os caches e o Terminal. Ele mantém o estado de navegação (lista atual, página, seleção) e decide quais telas mostrar com base nas ações do usuário.

Depende:

- **Terminal:** Para atualizar a interface visual
- **Cache de Listas:** Para recuperar dados de listas de discussão
- **Cache de Feed:** Para recuperar metadados de patches
- **Cache de Patches:** Para recuperar conteúdo de patches
- **Render:** Para renderizar conteúdo de patches
- **Log:** Para registrar operações de UI

Mensagens:

- `ShowLists`: Exibe a visão de listas de discussão na página especificada
- `ShowFeed`: Exibe o feed de patches para uma lista específica na página especificada
- `ShowPatch`: Exibe o conteúdo renderizado de um patch específico com título, lista e message ID
- `UpdateSelection`: Atualiza o item selecionado na visão atual (sem retorno)
- `PreviousPage`: Navega para a página anterior na visão atual
- `NextPage`: Navega para a próxima página na visão atual

- **NavigateBack:** Navega de volta na hierarquia (patch → feed → listas; nas listas, encerra a aplicação)
- **SubmitSelection:** Submete a seleção atual, retornando uma ação de navegação (abre feed, abre patch, ou encerra)
- **GetState:** Recupera o estado atual da UI

Render

O ator Render é responsável por processar conteúdo bruto de patches e convertê-los em uma representação visual formatada. Ele suporta múltiplos backends de renderização, incluindo ferramentas externas como bat ou delta.

Depende:

- **Shell:** Para executar programas externos de renderização
- **Configurações:** Para obter a configuração do renderizador a ser usado

Mensagens:

- **Render:** Renderiza o conteúdo bruto do patch usando o renderizador configurado

Cache

A fim de agilizar o acesso a dados diminuindo a quantidade de operações que dependam de chamadas de API ou do sistema de arquivos, será feito o uso de técnicas de *caching*.

Para cada domínio de dados, será criado um ator específico que se encarregará de gerenciar o cache para esse domínio. Sendo responsável por lidar com *cache misses* e *cache hits*, assim como com a validação de cache, garantindo consistência dos dados realizando o mínimo de acessos necessários a sistemas externos.

Listas de Discussão

Armazena em cache a lista de todas as listas de discussão disponíveis da API do Lore. Ele busca a lista completa, ordena alfabeticamente e a persiste no sistema de arquivos para acelerar inicializações subsequentes da aplicação. Ele gerencia validação de cache para atualizar periodicamente a lista.

Depende:

- **Lore API:** Para buscar os dados da lista de discussão
- **Sistema de Arquivos:** Para persistir o cache em disco
- **Configurações:** Para obter o caminho do arquivo de cache

Mensagens:

- **Get:** Recupera uma única lista de discussão por seu índice na lista ordenada
- **GetSlice:** Recupera um intervalo de listas de discussão para paginação
- **Refresh:** Força uma atualização completa do cache da API

- **Invalidate:** Limpa o cache em memória e em disco
- **IsAvailable:** Verifica se um intervalo de índices está disponível em cache
- **Len:** Retorna o número total de listas de discussão em cache
- **Persist:** Persiste o cache no sistema de arquivos
- **Load:** Carrega o cache do sistema de arquivos

Feed

Fornece cache por lista de discussão de metadados de patches (como autor, assunto e data). Ele busca feeds de patches sob demanda, suporta paginação e persiste os metadados no sistema de arquivos. Ele usa uma estratégia de validação inteligente para buscar apenas patches novos quando um feed foi atualizado no servidor.

Depende:

- **Lore API:** Para buscar metadados de patches
- **Sistema de Arquivos:** Para persistir o cache em disco
- **Configurações:** Para obter o caminho do diretório de cache
- **Log:** Para registrar operações de cache

Mensagens:

- **Get:** Recupera metadados de um único patch por seu índice dentro de um feed de lista de discussão
- **GetSlice:** Recupera um intervalo de metadados de patches para paginação
- **Refresh:** Atualiza inteligentemente o cache para uma lista de discussão específica, buscando apenas itens novos
- **Invalidate:** Limpa o cache para uma lista de discussão específica
- **IsAvailable:** Verifica se um intervalo de índices está disponível em cache para uma lista
- **Len:** Retorna o número de entradas de metadados em cache para uma lista
- **Persist:** Persiste o cache de uma lista no sistema de arquivos
- **Load:** Carrega o cache de uma lista do sistema de arquivos
- **IsLoaded:** Verifica se o cache de uma lista foi carregado do disco

Patches

Armazena em cache o conteúdo bruto de patches individuais (formato `.mbox`). Como o conteúdo do patch é imutável, uma vez que um patch é buscado e armazenado em cache, ele é considerado válido para sempre. Este ator armazena cada patch em um arquivo separado no disco e usa um buffer LRU (Least Recently Used) em memória para fornecer acesso rápido a patches visualizados recentemente.

Depende:

- **Lore API:** Para buscar o conteúdo bruto do patch
- **Sistema de Arquivos:** Para armazenar arquivos de patch permanentemente
- **Configurações:** Para obter o caminho do diretório de cache
- **Log:** Para registrar operações de cache

Mensagens:

- **Get:** Recupera o conteúdo bruto de um patch dado sua lista de discussão e ID de mensagem (busca da API se não estiver em cache)
- **Invalidate:** Remove um patch específico do cache em disco e buffer em memória
- **IsAvailable:** Verifica se um patch está em cache sem buscá-lo

App

O ator App atua como coordenador central da aplicação, gerenciando o ciclo de vida e coordenando a comunicação entre todos os outros atores. Ele é responsável por inicializar a aplicação, processar eventos de UI do Terminal e realizar encerramento gracioso quando apropriado.

Depende:

- **Todos os atores:** O App depende de praticamente todos os outros atores da aplicação como dependências injetadas

Mensagens:

- **Shutdown:** Inicia o processo de encerramento graceful da aplicação

Note que os eventos de teclado são registrados pelo Terminal e processados pelo App. Todavia isto não é feito com uma mensagem enviada pelo Terminal para o App. Mas sim o App que vai solicitar os eventos de UI ao Terminal no momento apropriado. Caso contrário, existiria um ciclo de dependências entre o Terminal e o App. Pois o App depende de Ui, que por sua vez depende de Terminal. Caso o Terminal tivesse de enviar mensagens ao App, então o Terminal dependeria do App, fechando o ciclo.

Capítulo 6

Conclusão

Apêndice A

Perguntas frequentes sobre o modelo

- **Não consigo decorar tantos comandos!**

Use a colinha que é distribuída juntamente com este modelo (gitlab.com/ccsl-usp/modelo-latex/raw/main/pre-compilados/colinha.pdf?inline=false).

- **Estou tendo problemas com caracteres acentuados.**

Versões modernas de $\text{\LaTeX}$ usam UTF-8, mas arquivos antigos podem usar outras codificações (como ISO-8859-1, também conhecido como latin1 ou Windows-1252). Nesses casos, use `\usepackage[latin1]{inputenc}` no preâmbulo do documento. Você também pode representar os caracteres acentuados usando comandos $\text{\LaTeX}$: `\a` para á, `\c{c}` para cedilha etc., independentemente da codificação usada no texto.¹

- **É possível resumir o nome das seções/capítulos que aparece no topo das páginas e no sumário?**

Sim, usando a sintaxe `\section[mini-titulo]{titulo enorme}`. Isso é especialmente útil nas legendas (*captions*) das figuras e tabelas, que muitas vezes são demasiadamente longas para a lista de figuras/tabelas.

- **Existe algum programa para gerenciar referências em formato bibtex?**

Sim, há vários. Uma opção bem comum é o JabRef; outra é usar Zotero ou Mendeley e exportar os dados deles no formato .bib.

- **Posso usar pacotes $\text{\LaTeX}$ adicionais aos sugeridos?**

Com certeza! Você pode modificar os arquivos o quanto desejar, o modelo serve só como uma ajuda inicial para o seu trabalho.

¹ Você pode consultar os comandos desse tipo mais comuns em en.wikibooks.org/wiki/LaTeX/Special_Characters. Observe que a dica sobre o pingo do i *não* é mais válida atualmente; basta usar `\i`.

Anexo A

As packages `imegoodies` e `imelooks`

Este modelo inclui as *packages* `imegoodies` e `imelooks`, que você pode querer usar em outros documentos $\text{\LaTeX}$.

`imegoodies` inclui um grande número de *packages* que são comumente usadas e bastante úteis. Em geral, você pode incluí-la em seus documentos sem que isso cause problemas de compatibilidade. Se, no entanto, algo não funcionar, você pode editar o arquivo para eliminar a *package* responsável pelo problema se ela não for necessária. `imegoodies` ainda inclui vários comentários explicativos sobre as *packages* carregadas.

`imelooks` também inclui um grande número de *packages*, mas estas são relacionadas mais explicitamente à aparência do documento (fontes, cores, margens etc.). Você também pode utilizá-la em outros documentos se quiser se aproximar da aparência deste modelo. `imelooks` reconhece diversos parâmetros que ativam/desativam aspectos específicos:

- `fonts` carrega as fontes deste modelo (`libertinus` e `sourcecodepro`), além de outros pequenos ajustes relacionados. Esta opção é sempre ativada por padrão; para desativá-la, use `nofonts`
- `spacing` utiliza os espaçamentos definidos neste modelo (margens, espaço entre parágrafos, indentação da primeira linha do parágrafo etc.). Esta opção é sempre ativada por padrão; para desativá-la, use `nospacing`
- `captions` e `footnotes` fazem respectivamente as legendas (das figuras e tabelas) e as notas de rodapé de acordo com este modelo. Estas opções são sempre ativadas por padrão; para desativá-las, use `nocaptions` e `nofootnotes`
- `autohttp` acrescenta o prefixo `http://` a URLs criadas com `url` que não incluam o *schema*. Esta opção é sempre ativada por padrão; para desativá-la, use `noautohttp`
- `hidelinks`, `borderlinks` e `colorlinks` definem a aparência dos hiperlinks. `hidelinks` faz os hiperlinks sem nenhuma formatação especial; `borderlinks` faz os hiperlinks serem envidos por um quadrado colorido (apenas na tela; o quadrado não é impresso); `colorlinks` faz o texto dos hiperlinks ser colorido. A opção `colorlinks` é sempre ativada por padrão

- `biblatex` carrega a *package biblatex* e os estilos bibliográficos deste modelo. Esta opção é sempre ativada por padrão; para desativá-la, use `nobiblatex`
- `raggedbib` faz a bibliografia (com `biblatex`) ser formatada com alinhamento à esquerda ao invés de justificado. Esta opção é sempre ativada por padrão, exceto quando o estilo bibliográfico é `plainnat-ime` (usado nas teses); para desativá-la, use `noraggedbib`; para ativá-la incondicionalmente, use `raggedbib`
- `bibstyle=?` seleciona um estilo bibliográfico específico. O estilo padrão é `numeric`, exceto em pôsteres e apresentações (`beamer-ime`) e *reports* (`plainnat-ime`)
- `listings` carrega a *package listings* e diversas configurações relacionadas usadas neste modelo. Esta opção é sempre ativada por padrão; para desativá-la, use `nolistings`
- `greeny`, `bluey`, `sandy` ativam esquemas de cores diferentes para pôsteres e apresentações (o padrão é `bluey`)
- `beamer` **desativa** algumas *packages* que são incompatíveis com a classe `beamer` (note que as opções `slides` e `presentation`, discutidas abaixo, já fazem isso)
- `presentation` (ou `slides`) e `poster` ativam as opções relevantes para, respectivamente, apresentações com `beamer` ou pôsteres com `tcolorbox`
- `report` ativa as opções relevantes para documentos com capítulos (cabeçalhos das páginas, características do sumário etc.)
- `thesis` ativa a opção `report` e também define o que é necessário para a geração da capa das teses de acordo com este modelo
- `resumoabstract` define os comandos `resumo` e `abstract` de acordo com este modelo. Esta opção é ativada por padrão com `report`; para desativá-la, use `noresumoabstract`
- `brazilian` verifica se a língua portuguesa está ativa no documento e, em caso negativo, gera um erro. Esta opção é ativada por padrão com a opção `thesis`; para desativá-la, use `nobrazilian`

Anexo B

Código-fonte e pseudocódigo

Com a *package listings*, programas podem ser inseridos diretamente no arquivo, como feito no caso do Programa ??, ou importados de um arquivo externo com o comando `\lstinputlisting`, como no caso do Programa B.1.

Programa B.1 Máximo divisor comum (arquivo importado).

```

1  FUNCTION euclid( $a, b$ )  $\triangleright$  The g.c.d. of  $a$  and  $b$ 
2       $r \leftarrow a \bmod b$ 
3      while  $r \neq 0$   $\triangleright$  We have the answer if  $r$  is 0
4           $a \leftarrow b$ 
5           $b \leftarrow r$ 
6           $r \leftarrow a \bmod b$ 
7      end
8      return  $b$   $\triangleright$  The g.c.d. is  $b$ 
9  end
```

Trechos de código curtos (menores que uma página) podem ou não ser incluídos como *floats*; trechos longos necessariamente incluem quebras de página e, portanto, não podem ser *floats*. Com *floats*, a legenda e as linhas separadoras são colocadas pelo comando `\begin{program}`; sem eles, utilize o ambiente `programruledcaption` (atenção para a colocação do comando `\label{}`, dentro da legenda), como no Programa B.2¹:

Programa B.2 Máximo divisor comum (em português).

```

1  FUNCAO euclides( $a, b$ )  $\triangleright$  O máximo divisor comum de  $a$  e  $b$ 
2       $r \leftarrow a \bmod b$ 
3  enquanto  $r \neq 0$   $\triangleright$  Atingimos a resposta se  $r$  é zero
4           $a \leftarrow b$ 
5           $b \leftarrow r$ 
```

cont $\longrightarrow$

¹ listings oferece alguns recursos próprios para a definição de *floats* e legendas, mas neste modelo não os utilizamos.

```

→ cont
6       $r \leftarrow a \bmod b$ 
7      fim
8  devolva  $b \triangleright$  O máximo divisor comum é  $b$ 
9  fim

```

Além do suporte às várias linguagens incluídas em listings, este modelo traz uma extensão para permitir o uso de pseudocódigo, útil para a descrição de algoritmos em alto nível. Ela oferece diversos recursos:

- Comentários seguem o padrão de C++ (`//` e `/* ... */`), mas o delimitador é impresso como “ $\triangleright$ ”.
- “`:=`”, “`<>`”, “`<=`”, “`>=`” e “`!=`” são substituídos pelo símbolo matemático adequado.
- É possível acrescentar palavras-chave além de “if”, “and” etc. com a opção “`morekeywords={pchave1,pchave2}`” (para um trecho de código específico) ou com o comando `\lstset{morekeywords={pchave1,pchave2}}` (como comando de configuração geral).
- É possível usar pequenos trechos de código, como nomes de variáveis, dentro de um parágrafo normal com `\lstinline{blah}`.
- “`$.$.`” ativa o modo matemático em qualquer lugar.
- Outros comandos $\text{\LaTeX}$ funcionam apenas em comentários; fora, a linguagem simula alguns pré-definidos (`\textit{}`, `\texttt{}` etc.).
- O comando `\label` também funciona em comentários; a referência correspondente (`\ref`) indica o número da linha de código. Se quiser usá-lo numa linha sem comentários, use `/// \label{blah}`; “`///`” funciona como `//`, permitindo a inserção de comandos $\text{\LaTeX}$, mas não imprime o delimitador ($\triangleright$).
- Para suspender a formatação automática, use `\noparse{blah}`.
- Para forçar a formatação de um texto como função, identificador, palavra-chave ou comentário, use `\func{blah}`, `\id{blah}`, `\kw{blah}` ou `\comment{blah}`.
- Palavras-chave dentro de comentários não são formatadas automaticamente; se necessário, use `\func{}`, `\id{}` etc. ou comandos $\text{\LaTeX}$ padrão.
- As palavras “Program”, “Procedure” e “Function” têm formatação especial e fazem a palavra seguinte ser formatada como função. Funções em outros lugares *não* são detectadas automaticamente; use `\func{}`, a opção “`functions={func1,func2}`” ou o comando “`\lstset{functions={func1,func2}}`” para que elas sejam detectadas.
- Além de funções, palavras-chave, strings, comentários e identificadores, há “`specialidentifiers`”. Você pode usá-los com `\specialid{blah}`, com a opção “`specialidentifiers={id1,id2}`” ou com o comando “`\lstset{specialidentifiers={id1,id2}}`”.

Referências

- [KWORKFLOW 2024a] KWORKFLOW. *KWorkflow*. 2024. URL: <https://kworkflow.org/> (acesso em 19/12/2024) (citado na pg. 34).
- [KWORKFLOW 2024b] KWORKFLOW. *patch-hub*. 2024. URL: <https://github.com/kworkflow/patch-hub> (acesso em 19/12/2024) (citado na pg. 34).
- [LINUX KERNEL DOCUMENTATION 2024a] LINUX KERNEL DOCUMENTATION. *Submitting patches*. 2024. URL: <https://docs.kernel.org/process/submitting-patches.html> (acesso em 19/12/2024) (citado na pg. 33).
- [LINUX KERNEL DOCUMENTATION 2024b] LINUX KERNEL DOCUMENTATION. *Subsystem APIs*. 2024. URL: <https://docs.kernel.org/subsystem-apis.html> (acesso em 19/12/2024) (citado na pg. 33).
- [TADOKORO 2023] David TADOKORO. *The lore.kernel.org API*. Set. de 2023. URL: <https://blog.kworkflow.org/the-lore.kernel.org-api/> (acesso em 19/12/2024) (citado na pg. 34).
- [WIKIPEDIA 2024a] WIKIPEDIA. *Linux*. 2024. URL: <https://pt.wikipedia.org/wiki/Linux> (acesso em 19/12/2024) (citado na pg. 33).
- [WIKIPEDIA 2024b] WIKIPEDIA. *Linux (núcleo)*. 2024. URL: [https://pt.wikipedia.org/wiki/Linux_\(n%C3%BAcleo\)](https://pt.wikipedia.org/wiki/Linux_(n%C3%BAcleo)) (acesso em 19/12/2024) (citado na pg. 33).

Índice remissivo

C

Captions, *veja* Legendas
Código-fonte, *veja* Floats

E

Equações, *veja* Modo matemático

F

Figuras, *veja* Floats
Floats
 Algoritmo, *veja* Floats, ordem
Fórmulas, *veja* Modo matemático

I

Inglês, *veja* Língua estrangeira

L

Legendas, [49](#)

M

Mendeley, [49](#)

P

Palavras estrangeiras, *veja* Língua estrangeira

R

Rodapé, notas, *veja* Notas de rodapé

S

Subcaptions, *veja* Subfiguras
Sublegendas, *veja* Subfiguras

T

Tabelas, *veja* Floats

V

Versão corrigida, *veja* Tese/Dissertação, versões
Versão original, *veja* Tese/Dissertação, versões

Z

Zotero, [49](#)